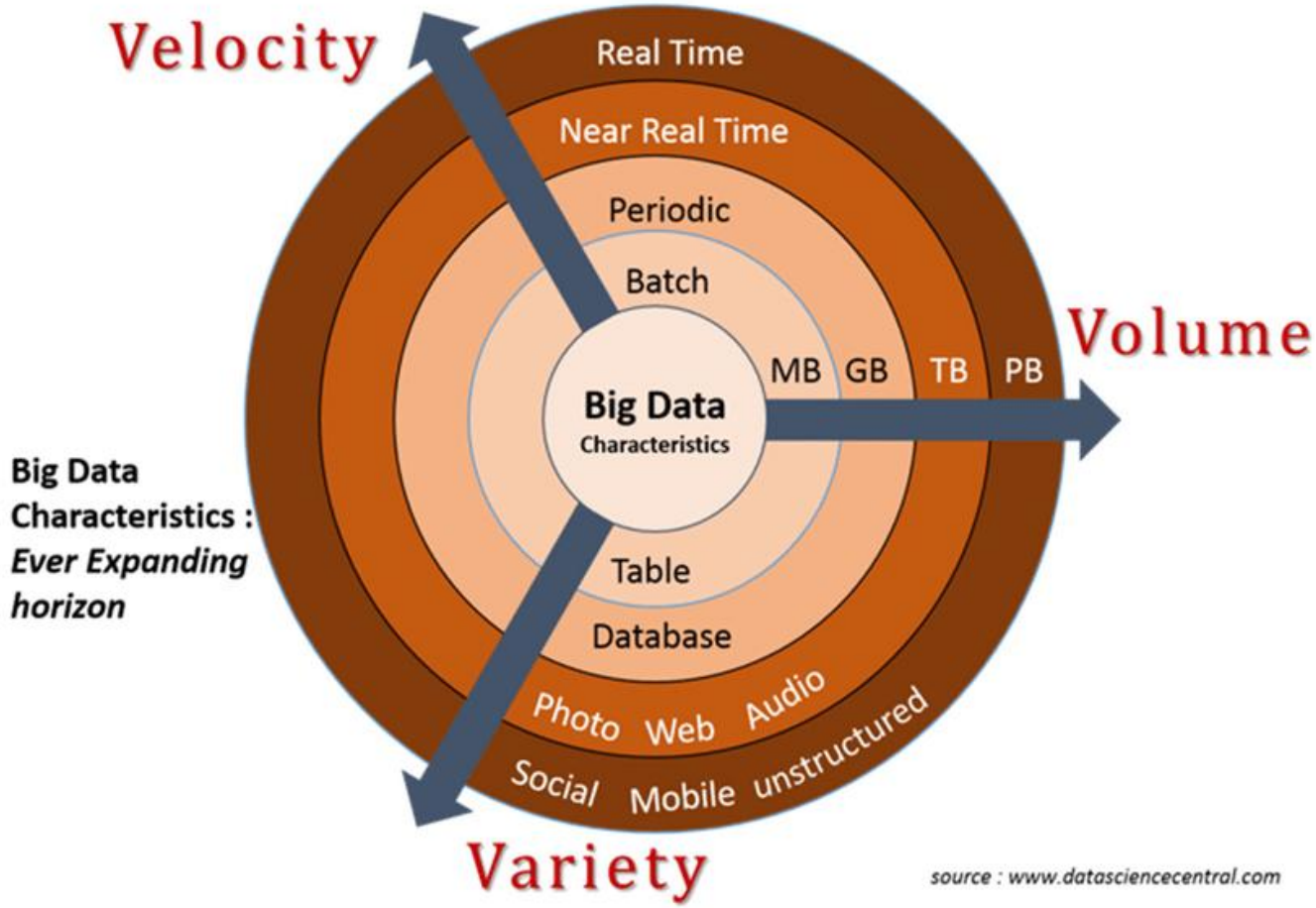
UNIT-1

Tabla de equivalencias de informática			
Medida	Simbología	Equivalencia	Equivalencia en Bytes
byte	b	8 bits	1 byte
kilobyte	Kb	1024 bytes	1024 bytes
megabyte	MB	1024 KB	1 048 576 bytes
gigabyte	GB	1024 MB	1 073 741 824 bytes
terabyte	TB	1024 GB	1 099 511 627 778 bytes
petabyte	PB	1024 TB	1 125 899 906 842 624 bytes
exabyte	EB	1024 PB	1 152 921 504 606 846 976 bytes
zetabyte	ZB	1024 EB	1 180 591 620 717 411 303 424 bytes
yottabyte	YB	1024 ZB	1 280 925 819 614 629 174 706 176 bytes
brontobyte	BB	1024 YB	1 237 940 039 285 380 274 899 124 224 bytes
geopbyte	GB	1024 BB	1 267 650 600 228 229 401 496 703 205 376 bytes

es.tableworld.net

What is Big data ?

Big data refers to data sets that are too large or complex to be dealt with by traditional data-processing application software.



Scalability Imperative: 3 "Vs" Drive Big Data Everywhere

Information from Everywhere

Radical Flexibility

Extreme Scalability

Volume

12 terabytes

of Tweets created daily

Velocity

5 million

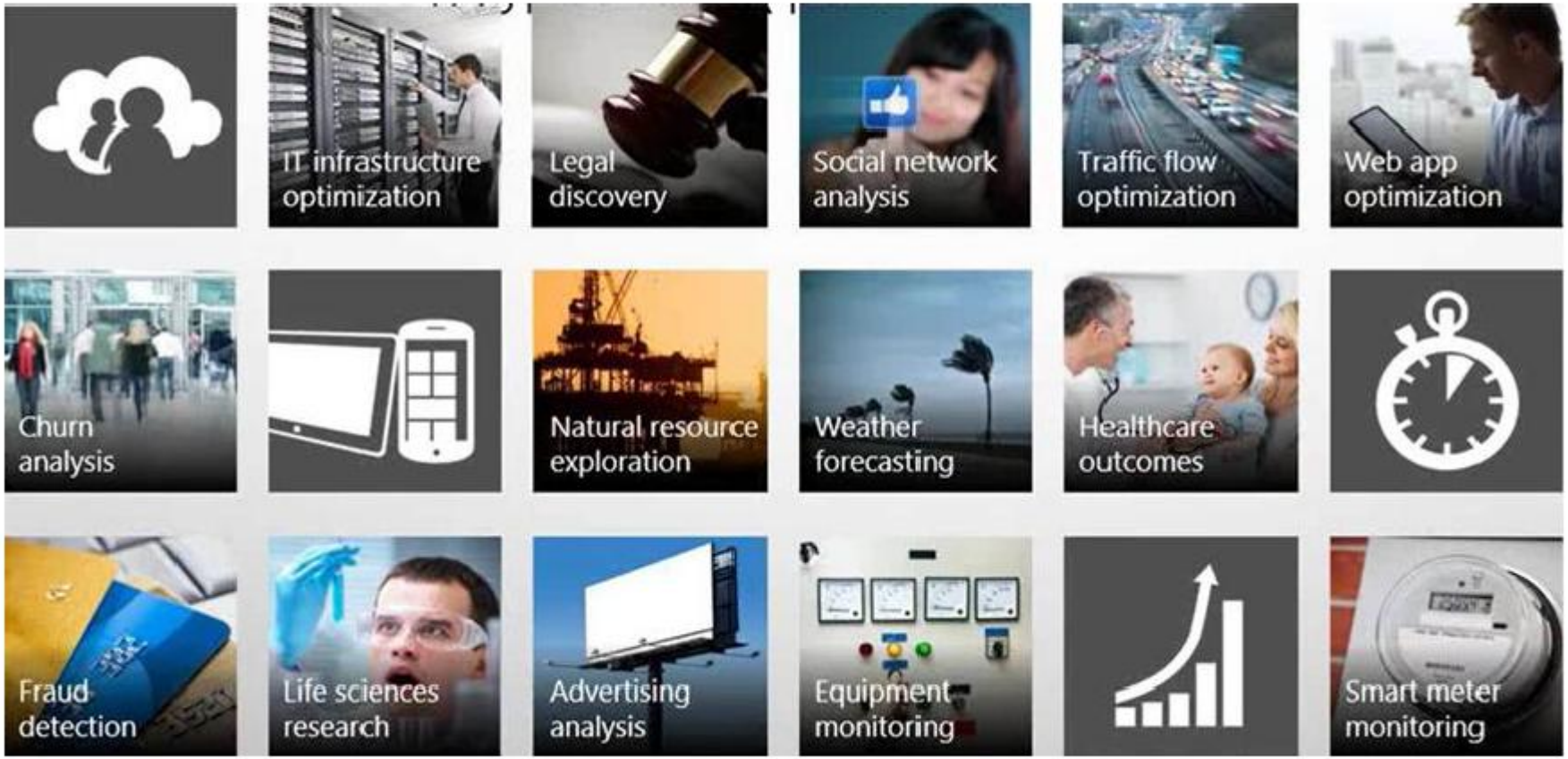
trade events per second

Variety

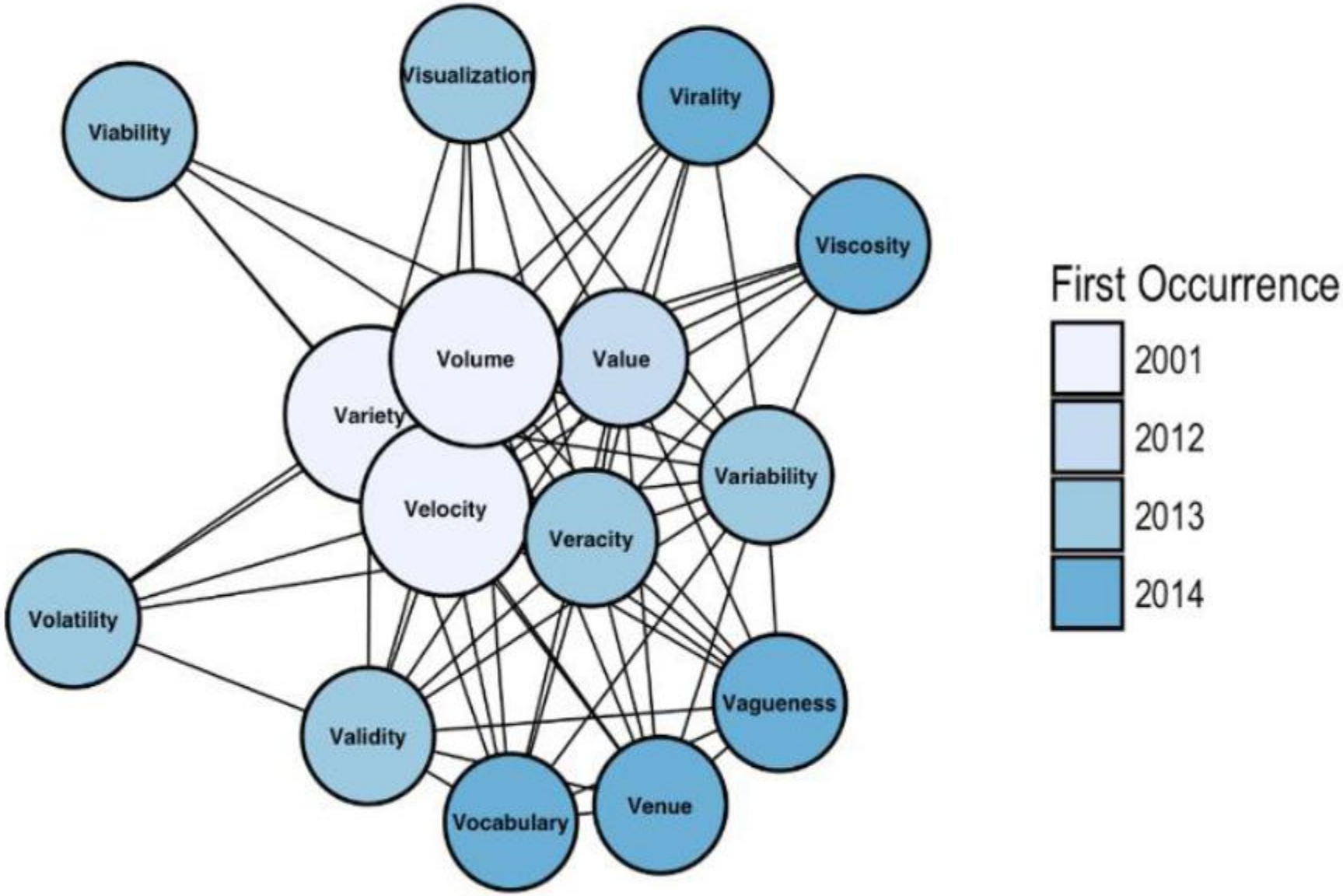
100's video feeds

from surveillance cameras

Common Big Data Customer Scenarios



15 Vs of Big Data till 2014



Updated 2017 list:

The 42 V's of Big Data and Data Science.

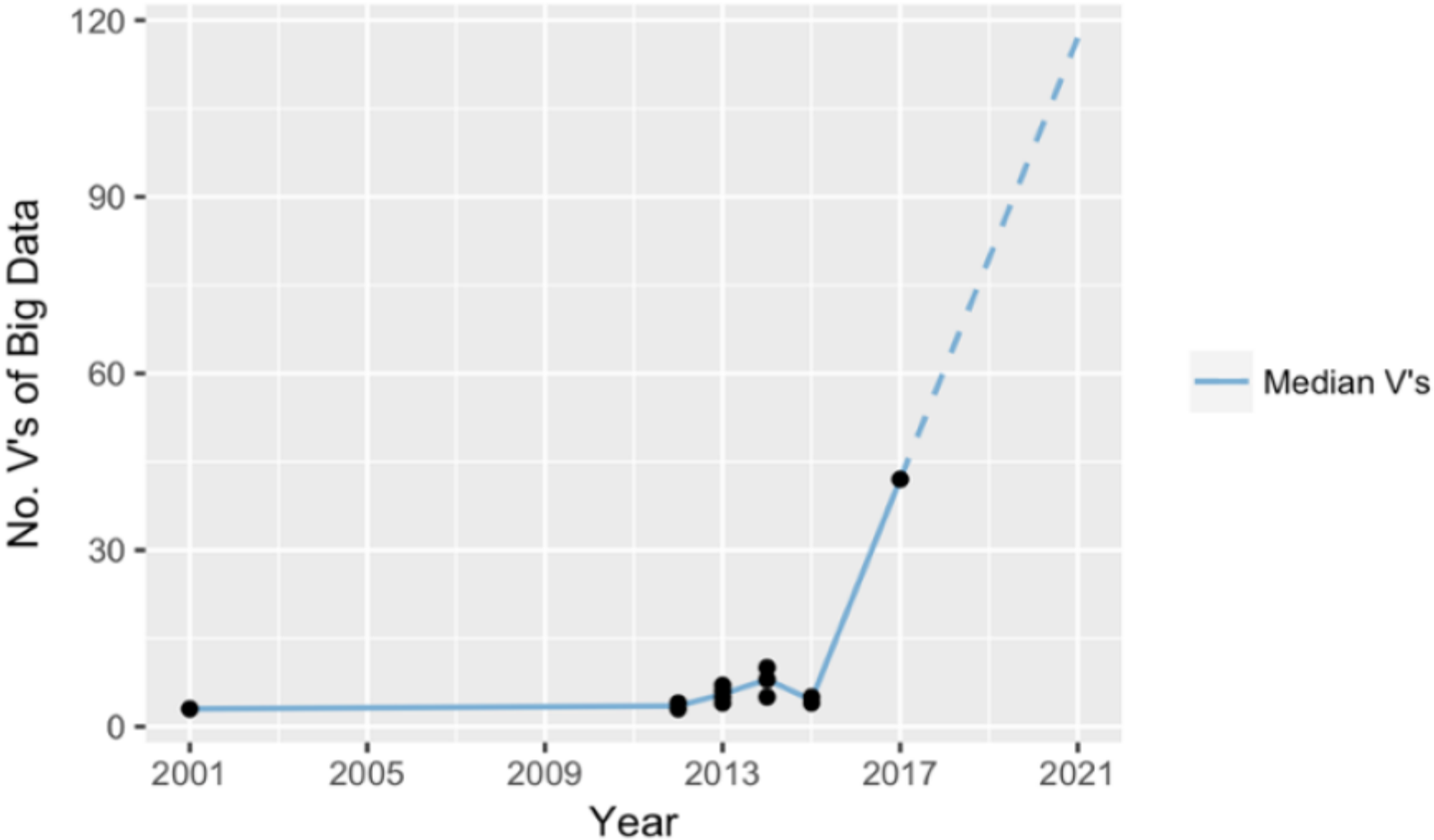
- **Vagueness:** The meaning of found data is often very unclear, regardless of how *much* data is available.
- **Validity:** Rigor in analysis (e.g., Target Shuffling) is essential for valid predictions.
- **Valor:** In the face of big data, we must gamely tackle the big problems.
- **Value:** Data science continues to provide ever-increasing value for users as more data becomes available and new techniques are developed.
- **Vane:** Data science can aid decision making by pointing in the correct direction.
- **Vanilla:** Even the simplest models, constructed with rigor, can provide value.
- **Vantage:** Big data allows us a privileged view of complex systems.
- **Variability:** Data science often models variable data sources. Models deployed into production can encounter especially wild data.
- **Variety:** In data science, we work with many data formats (flat files, relational databases, graph networks) and varying levels of data completeness.
- **Varifocal:** Big data and data science together allow us to see both the forest *and* the trees.

- **Varmint:** As big data gets bigger, so can software bugs!
- **Varnish:** How end-users interact with our work matters, and polish counts.
- **Vastness:** With the advent of the Internet of Things (IoT), the "bigness" of big data is accelerating.
- **Vaticination:** Predictive analytics provides the ability to forecast. (Of course, these forecasts can be more or less accurate depending on rigor and the complexity of the problem. The future is pesky and never conforms to our March Madness brackets.)
- **Vault:** With many data science applications based on large and often sensitive data sets, data security is increasingly important.
- **Veer:** With the rise of agile data science, we should be able to navigate the customer's needs and change directions quickly when called upon.
- **Veil:** Data science provides the capability to peer behind the curtain and examine the effects of latent variables in the data.
- **Velocity:** Not only is the volume of data ever increasing, but the rate of data generation (from the Internet of Things, social media, etc.) is increasing as well.
- **Venue:** Data science work takes place in different locations and under different arrangements: Locally, on customer workstations, and in the cloud.
- **Veracity:** Reproducibility is essential for accurate analysis.

- **Verdict:** As an increasing number of people are affected by models' decisions, Veracity and Validity become ever more important.
- **Versed:** Data scientists often need to know a little about a great many things: mathematics, statistics, programming, databases, etc.
- **Version Control:** You're using it, right?
- **Vet:** Data science allows us to vet our assumptions, augmenting intuition with evidence.
- **Vexed:** Some of the excitement around data science is based on its potential to shed light on large, complicated problems.
- **Viability:** It is difficult to build robust models, and it's harder still to build systems that will be **viable in production**.
- **Vibrant:** A thriving data science community is vital, and it provides insights, ideas, and support in all of our endeavors.
- **Victual:** Big data — the food that fuels data science.
- **Viral:** How does **data spread** among other users and applications?
- **Virtuosity:** If data scientists need to know a little about many things, we should also grow to know a lot about one thing.
- **Viscosity:** Related to Velocity; how difficult is the data to work with?

- **Visibility:** Data science provides visibility into complex big data problems.
- **Visualization:** Often the *only* way customers interact with models.
- **Vivify:** Data science has the potential to animate all manner of decision making and business processes, from marketing to fraud detection.
- **Vocabulary:** Data science provides a vocabulary for addressing a variety of problems. Different modeling approaches tackle different problem domains, and different validation techniques harden these approaches in different applications.
- **Vogue:** "Machine Learning" becomes "Artificial Intelligence", which becomes...?
- **Voice:** Data science provides the ability to speak with knowledge (though not *all* knowledge, of course) on a diverse range of topics.
- **Volatility:** Especially in production systems, one has to prepare for data volatility. Data that should "never" be missing suddenly disappears, numbers suddenly contain characters!
- **Volume:** More people use data-collecting devices as more devices become internet-enabled. The volume of data is increasing at a staggering rate.
- **Voodoo:** Data science and big data aren't voodoo, but how can we convince potential customers of data science's value to deliver results with real-world impact?
- **Voyage:** May we always keep learning as we tackle the problems that data science provides.
- **Vulpine:** Nate Silver would like you to be a fox, please.

These nine distinct sets encompass fifteen different "V's," orbiting the original three. We can safely say we are now **well on the way** to 100 V's of Big Data and Data Science!, by Tom Shafer



Why DFS?

1 Server
- 4 I/O Channels
- Each Channel – 100 MB/S

44 minutes (around)

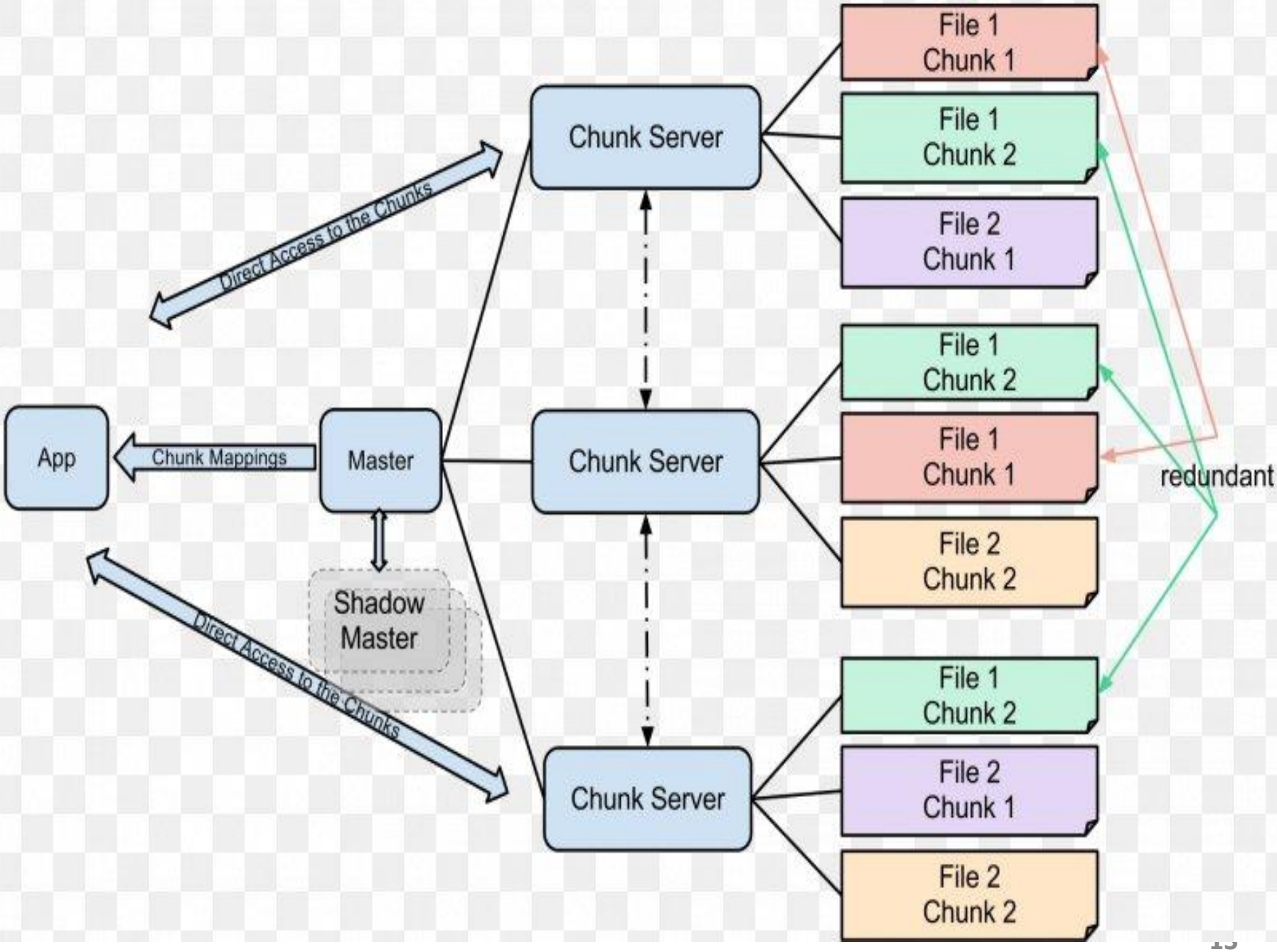
1 0 Servers
- 4 I/O Channels
- Each Channel – 100 MB/S

4.4 minutes (around)

Till today disk I/O channels are still the constraint , DFS overcomes the constraint by logically grouping multiple small machines to behave as one big machine.

Google File System

- GFS is a distributed file system built for large, distributed data-intensive applications like Gmail or YouTube.
- Originally, it was built to store data generated by Google's large crawling and indexing system.
- GFS is a scalable distributed file system developed by Google for its large data-intensive applications.
- GFS was built for handling batch processing on large data sets and is designed for system-to-system interaction, not user-to-system interaction.
- GFS is not implemented in the kernel of an operating system, but is instead provided as a user space library.
- GFS provides fault tolerance, reliability, scalability, availability and performance to large networks and connected nodes.


File 1
Chunk 1
File 1
Chunk 2
File 2
Chunk 1
Chunk Server
Direct Access to the Chunks
File 1
Chunk 2
File 1
Chunk 1
File 2
Chunk 2
App
Chunk Mappings
Master
Chunk Server
redundant
Shadow
Master
Direct Access to the Chunks
File 1
Chunk 2
File 2
Chunk 1
File 2
Chunk 2
Chunk Server

1.Client uses GFS client library

2.Communicates with:

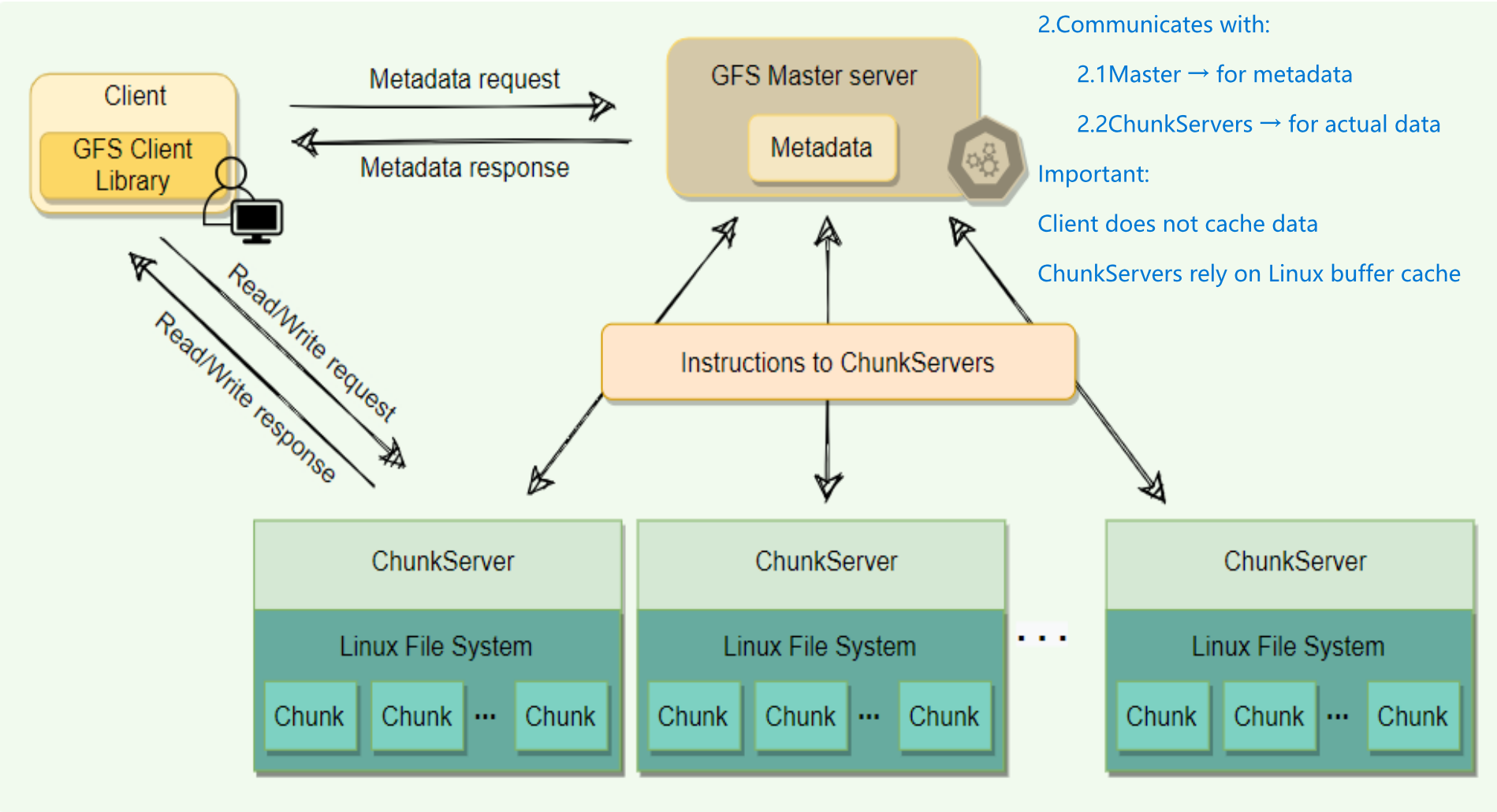
2.1Master → for metadata

2.2ChunkServers → for actual data

Important:

Client does not cache data

ChunkServers rely on Linux buffer cache



GFS high-level architecture

Chunks

- A GFS cluster consists of a single master and multiple ChunkServers and is accessed by multiple clients.

- GFS breaks files into multiple fixed-size chunks where each chunk is 64 megabytes in size.

Chunk handle

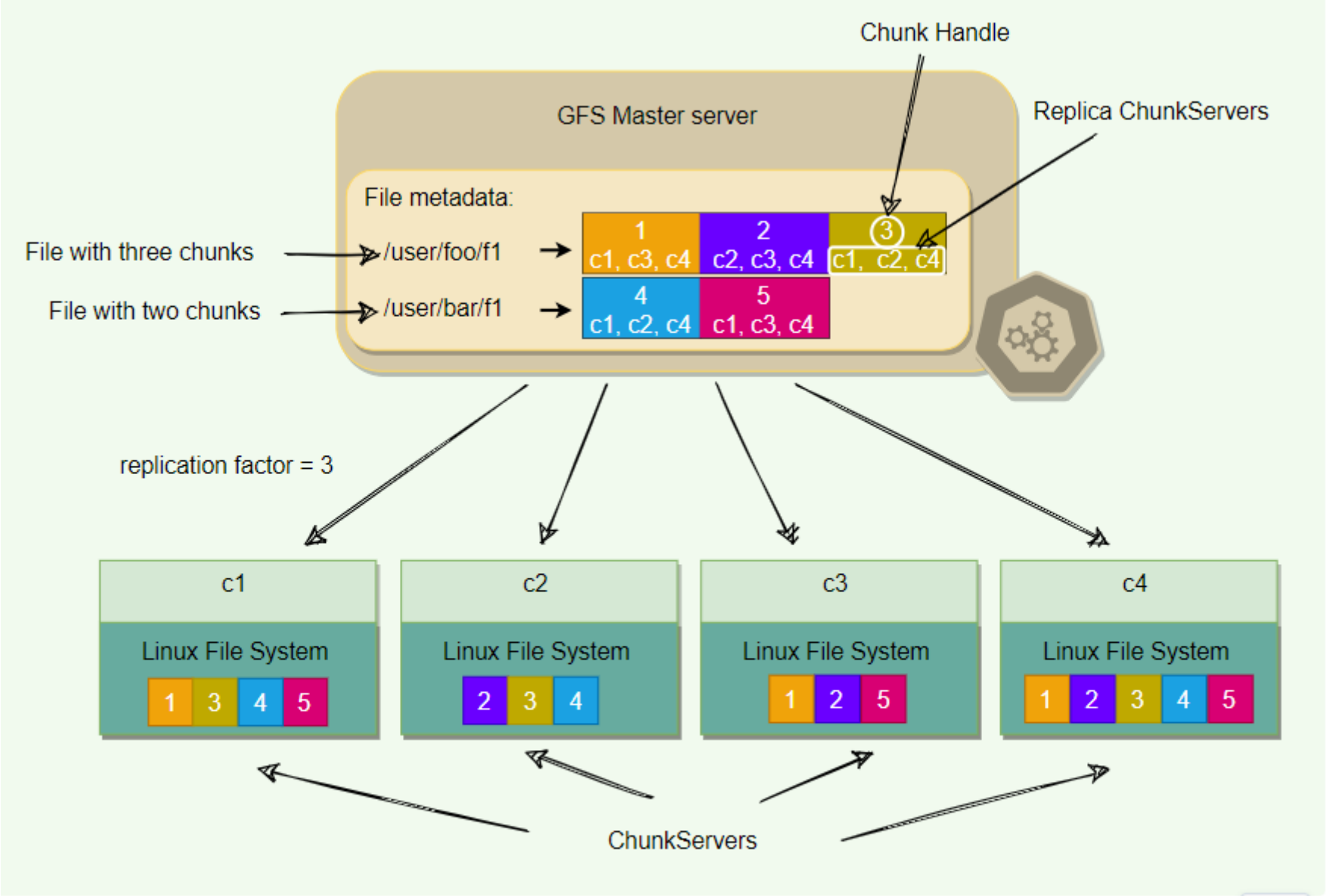
- Each chunk is identified by an immutable and globally unique 64-bit ID number called chunk handle. This allows for unique chunks. If each chunk is 64 MB, total storage space would be more than exa-bytes.

Cluster

- GFS is organized into a simple network of computers called a cluster. All GFS clusters contain three kinds of entities:
  - A single master server
  - Multiple ChunkServers
  - Many clients
- The master stores all metadata about the system, while the Chunk Servers store the real file data.

ChunkServer

- ChunkServers store chunks on local disks as regular Linux files and read or write chunk data specified by a chunk handle and byte-range.
- For reliability, each chunk is replicated to multiple ChunkServers. By default, GFS stores three replicas, though different replication factors can be specified on a per-file basis.



Chunk replication

Master

- Master server is the coordinator of a GFS cluster and is responsible for keeping track of filesystem metadata:
- The metadata stored at the master includes:
  - Name and directory of each file
  - Mapping of each file to its chunks
  - Current locations of chunks
  - Access control information
- The master also controls system-wide activities such as chunk lease management (locks on chunks with expiration), garbage collection of orphaned chunks, and chunk migration between ChunkServers. Master assigns chunk-handle to chunks at the time of chunk creation.
- The master periodically communicates with each ChunkServer in HeartBeat messages to give it instructions and collect its state.

Master

- For performance and fast random access, all metadata is stored in the master's main memory. This includes the entire filesystem namespace as well as all the name-to-chunk mappings.
- For fault tolerance and to handle a master crash, all metadata changes are written to the disk onto an operation log. This operation log is also replicated onto remote machines. The operation log is similar to a journal. Every operation to the file system is logged into this file.
- The master is a single point of failure, hence, it replicates its data onto several remote machines so that the master can be readily restored on failure

Client

- Client is an entity that makes a read or write request to GFS.
- GFS client library is linked into each application that uses GFS. This library communicates with the master for all metadata-related operations like creating or deleting files, looking up files, etc.
- To read or write data, the client interacts directly with the ChunkServers that hold the data.
- Neither the client nor the ChunkServer caches file data. Client caches offer little benefit because most applications stream through huge files or have working sets too large to be cached.
- ChunkServers rely on the buffer cache in Linux to maintain frequently accessed data in memory.

Google built GFS keeping the following goals in mind:

Scalable: GFS should run reliably on a very large system built from commodity hardware.

Fault-tolerant: The design must be sufficiently tolerant of hardware and software failures to enable application-level services to continue their operation in the face of any likely combination of failure conditions.

Large files: Files stored in GFS will be huge. Multi-GB files are common. Large sequential and small random reads.

Sequential writes: The workloads also have many large, sequential writes that append data to files. Typical operation sizes are similar to those for reads. Once written, files are seldom modified again.

Not optimized for small data: Small, random reads and writes do occur and are supported, but the system is not optimized for such cases

Concurrent access: The level of concurrent access will also be high, with large numbers of concurrent appends being particularly prevalent, often accompanied by concurrent reads.

High throughput: GFS should be optimized for high and sustained throughput in reading the data, and this is prioritized over latency. This is not to say that latency is unimportant; rather, GFS needs to be optimized for high-performance reading and appending large volumes of data for the correct operation of the system.

DFS or Distributed File System:

- Distributed File System is a file system that allows us to store data over multiple nodes or machines in a cluster and allows multiple users to access data.
- So basically, it serves the same purpose as the file system which is available in your machine, like for windows you have NTFS (New Technology File System) or for Mac you have HFS (Hierarchical File System).
- The only difference is that, in case of Distributed File System, you store data in multiple machines rather than single machine.
- Even though the files are stored across the network, DFS organizes, and displays data in such a manner that a user sitting on a machine will feel like all the data is stored in that very machine.

What is HDFS?

- Hadoop Distributed file system or HDFS is a Java based distributed file system that allows you to store large data across multiple nodes in a Hadoop cluster.

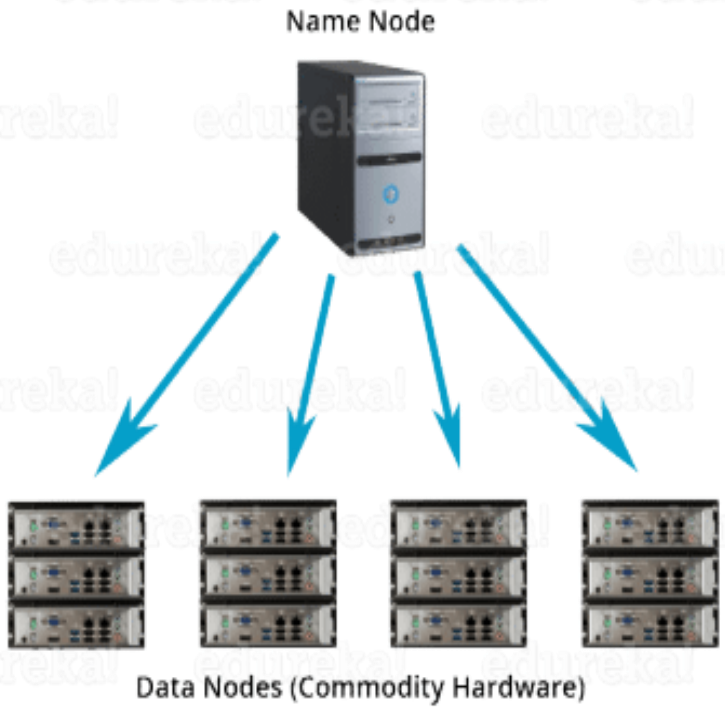
Advantages Of HDFS

- Distributed Storage
- Distributed & Parallel Computation
- Horizontal Scalability

Distributed Storage

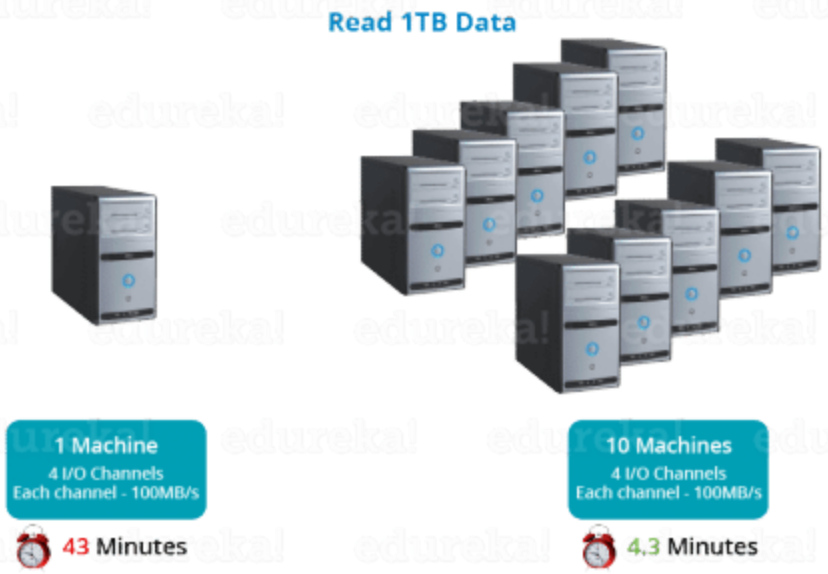
- When you access Hadoop Distributed file system from any of the ten machines in the Hadoop cluster, you will feel as if you have logged into a single large machine which has a storage capacity of 10 TB (total storage over ten machines).

- It means that you can store a single large file of 10 TB which will be distributed over the ten machines (1 TB each). So, it is **not limited to the physical boundaries** of each individual machine.



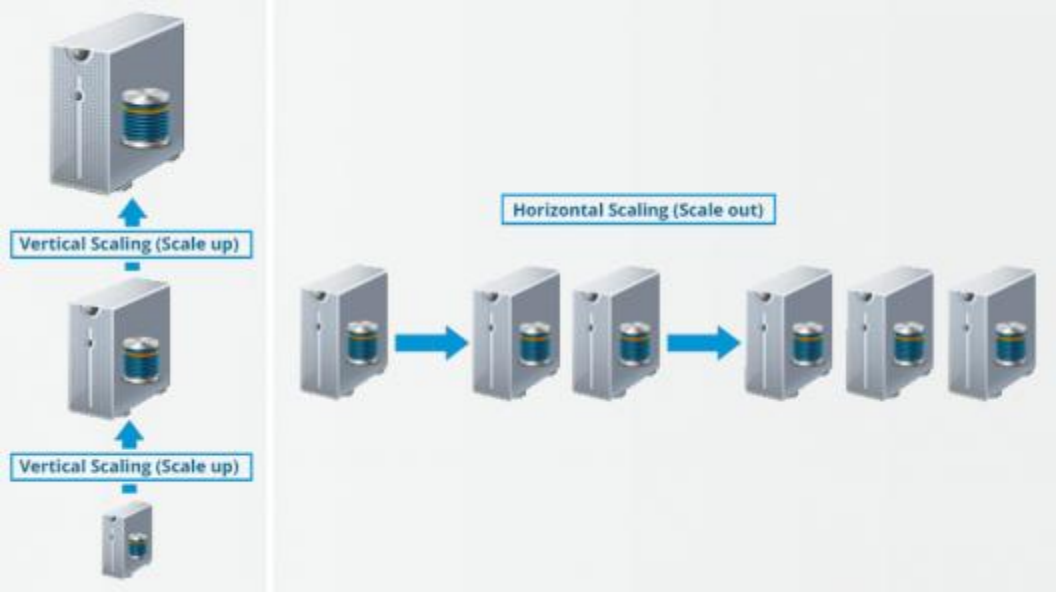
Distributed & Parallel Computation

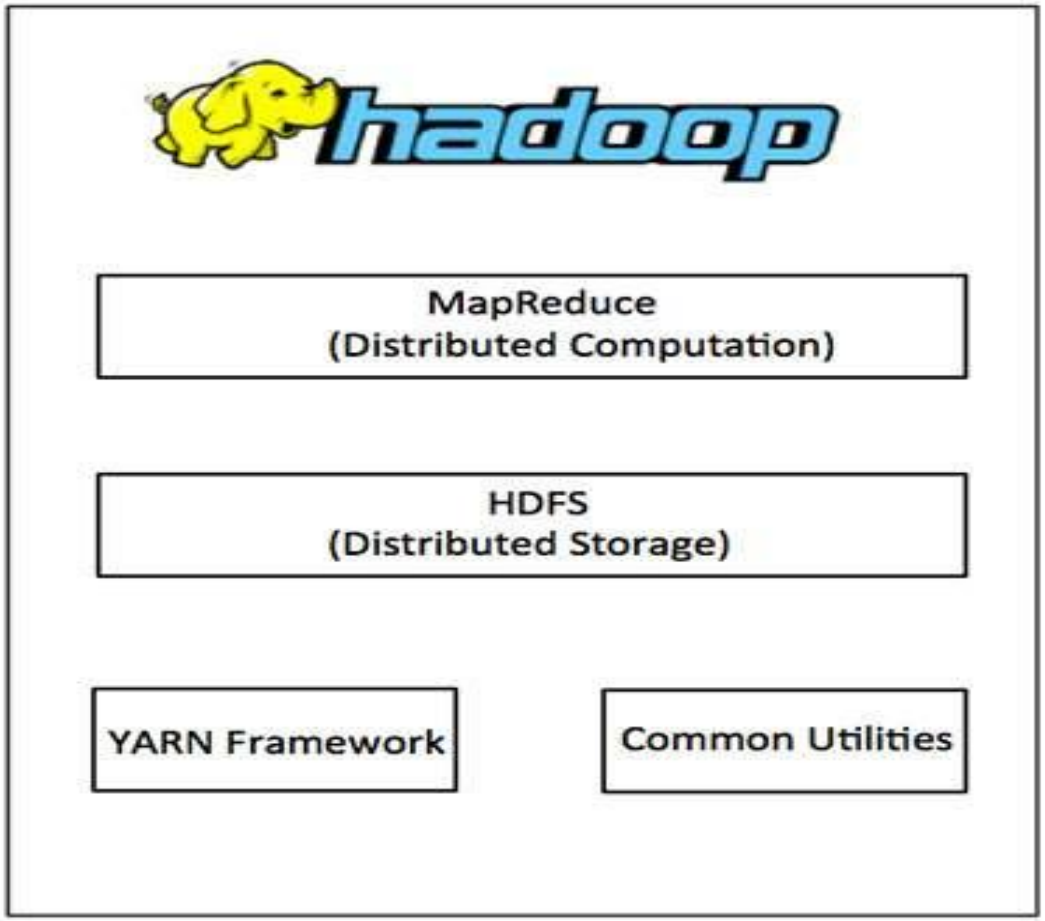
- Because the data is divided across the machines, it allows us to take advantage of **Distributed and Parallel Computation**.
- Each of the nodes is working with a part of the 1 TB file in parallel.
- Therefore, the work which was taking 43 minutes before, gets finished in just 4.3 minutes now as the work got divided over ten machines.

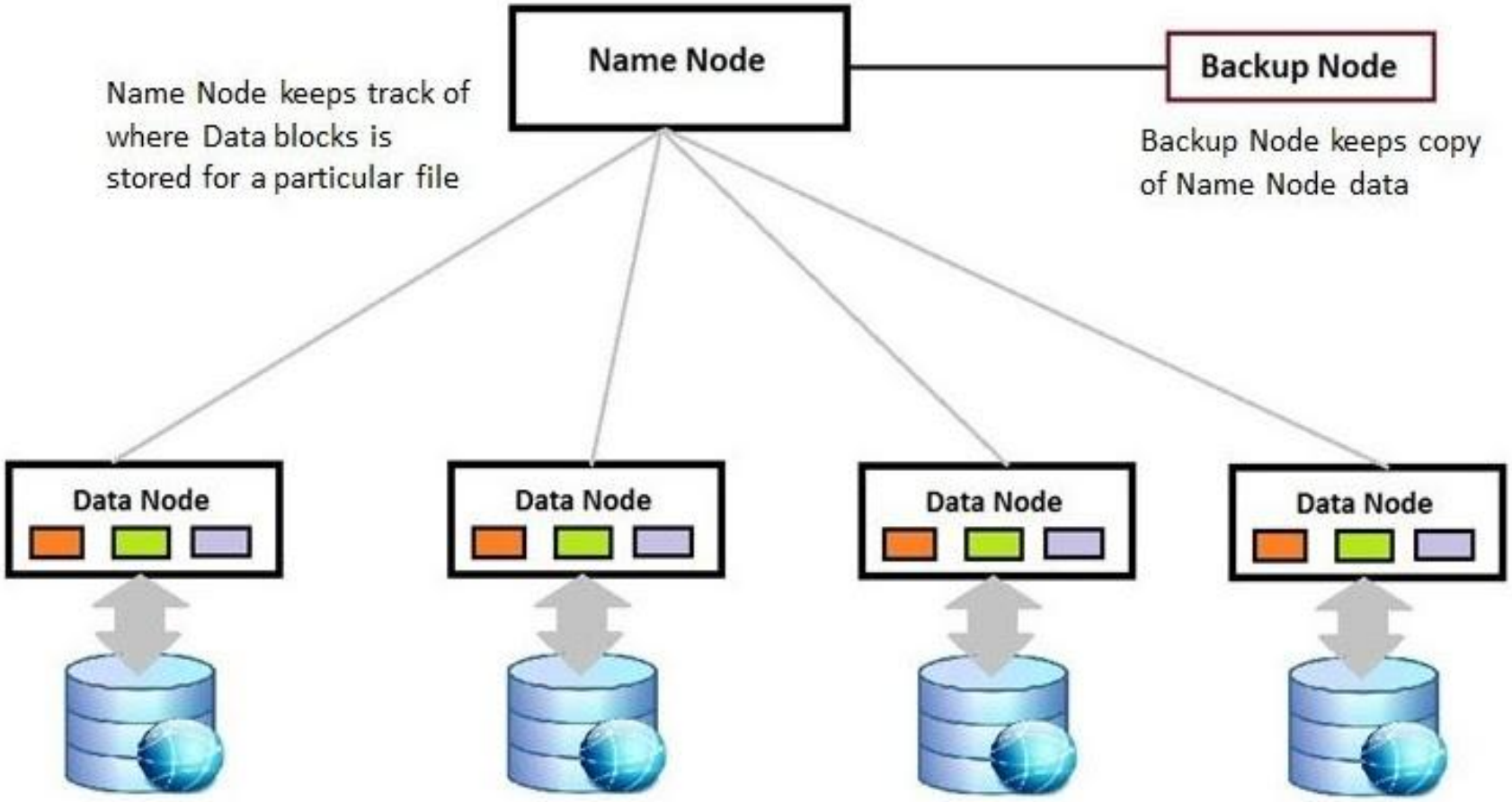


Horizontal Scalability

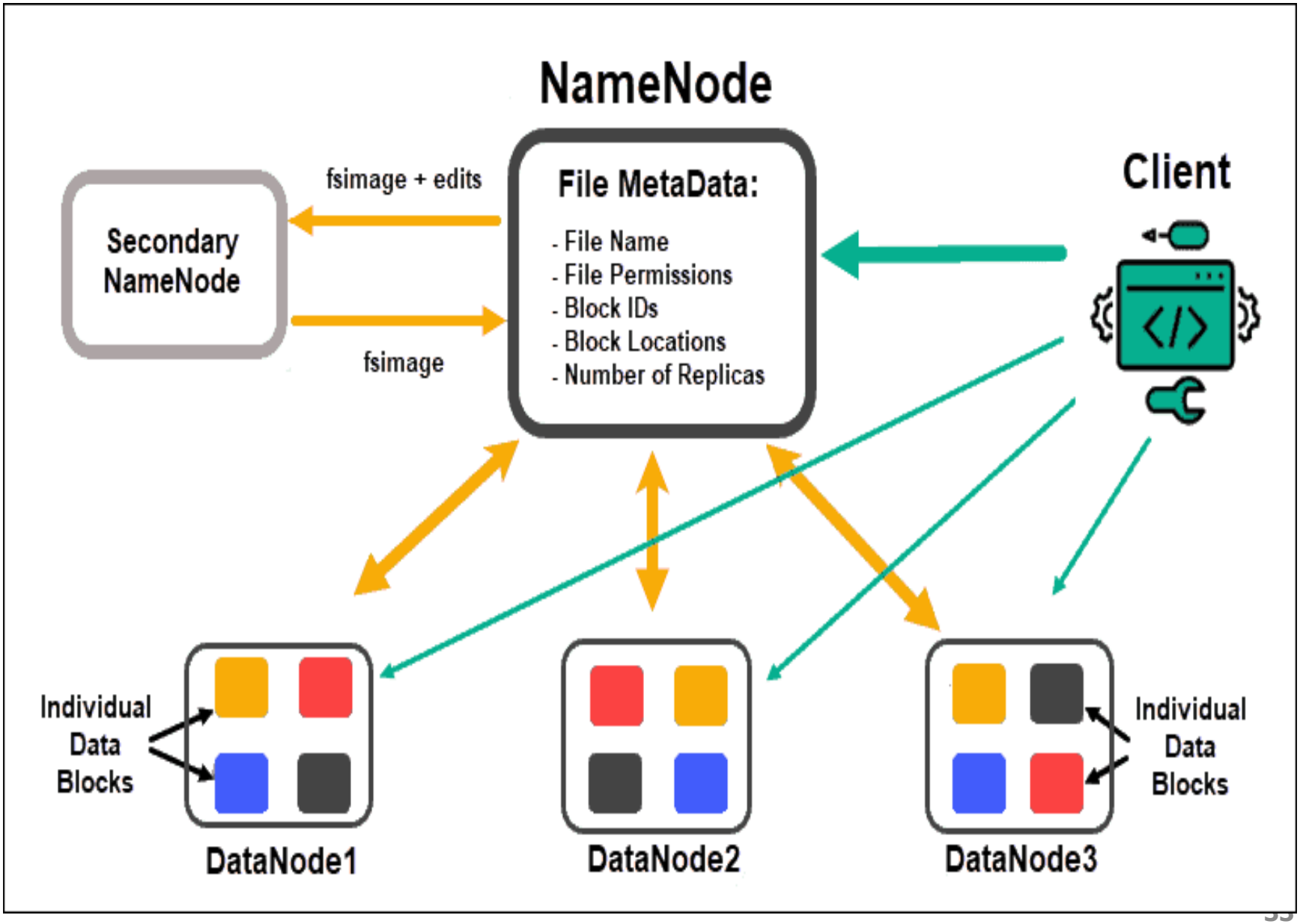
- In case of **horizontal scaling (scale out)**, you add more nodes to existing cluster instead of increasing the hardware capacity of individual machines.
- And most importantly, you can **add more machines on the go** i.e. Without stopping the system**.**
- Therefore, while scaling out we don't have any down time or green zone, nothing of such sort. At the end of the day, you will have more machines working in parallel to meet your requirements.




hadoop
MapReduce
(Distributed Computation)
HDFS
(Distributed Storage)
YARN Framework
Common Utilities


Name Node
Backup Node
Name Node keeps track of where Data blocks is stored for a particular file
Backup Node keeps copy of Name Node data
Data Node
Data Node
Data Node
Data Node

Hadoop Distributed File System (HDFS) cluster



- The **Hadoop Distributed File System (HDFS)** is fault-tolerant by design.
- Data is stored in individual data blocks in three separate copies across multiple nodes and server racks.
- If a node or even an entire rack fails, the impact on the broader system is negligible.

✓ **DataNodes** process and store data blocks

✓ **NameNodes** maintain data block metadata, and control client access.

DataNode

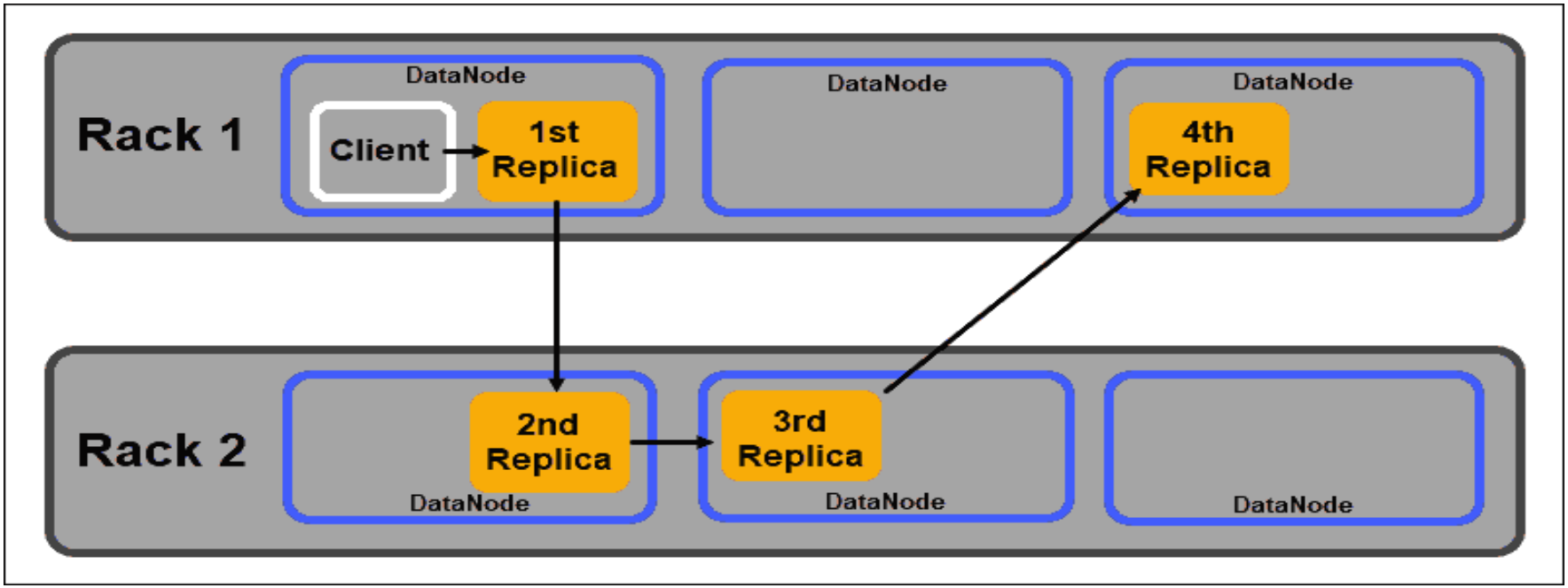
- Each DataNode in a cluster uses a background process to store the individual blocks of data on slave servers. By default, HDFS stores three copies of every data block on separate DataNodes.

- The NameNode uses a rack-aware placement policy. This means that the DataNodes that contain the data block replicas cannot all be located on the same server rack.

- A DataNode communicates and accepts instructions from the NameNode roughly twenty times a minute. Also, it reports the status and health of the data blocks located on that node once an hour. Based on the provided information, the NameNode can request the DataNode to create additional replicas, remove them, or decrease the number of data blocks present on the node.

Rack Aware Placement Policy

- One of the main objectives of a distributed storage system like HDFS is to maintain high availability and replication. Therefore, data blocks need to be distributed not only on different DataNodes but on nodes located on different server racks.
- This ensures that the failure of an entire rack does not terminate all data replicas. The HDFS NameNode maintains a default rack-aware replica placement policy:
- **The first data block replica is placed on the same node as the client.**
- **The second replica is automatically placed on a random DataNode on a different rack.**
- **The third replica is placed in a separate DataNode on the same rack as the second replica.**
- Any additional replicas are stored on random DataNodes throughout the cluster.
- This rack placement policy maintains only one replica per node and sets a limit of two replicas per server rack.



Rack failures are much less frequent than node failures. HDFS ensures high reliability by always storing at least one data block replica in a DataNode on a different rack.

Data Reliability and Fault Toleration

- ***Heartbeat*** is a recurring TCP handshake signal. DataNodes, located on each slave server, continuously send a heartbeat to the NameNode located on the master server. The default heartbeat time-frame is three seconds. If the NameNode does not receive a signal for more than ten minutes, it writes the DataNode off, and its data blocks are auto-scheduled on different nodes.
- Do not lower the heartbeat frequency to try and lighten the load on the NameNode. Keeping NameNodes 'informed' is crucial, even in extremely large clusters. Without a regular and frequent heartbeat influx, the NameNode is severely hampered and cannot control the cluster as effectively.
- To avoid serious fault consequences, keep the default rack awareness settings and store replicas of data blocks across server racks. If you lose a server rack, the other replicas survive, and the impact on data processing is minimal.

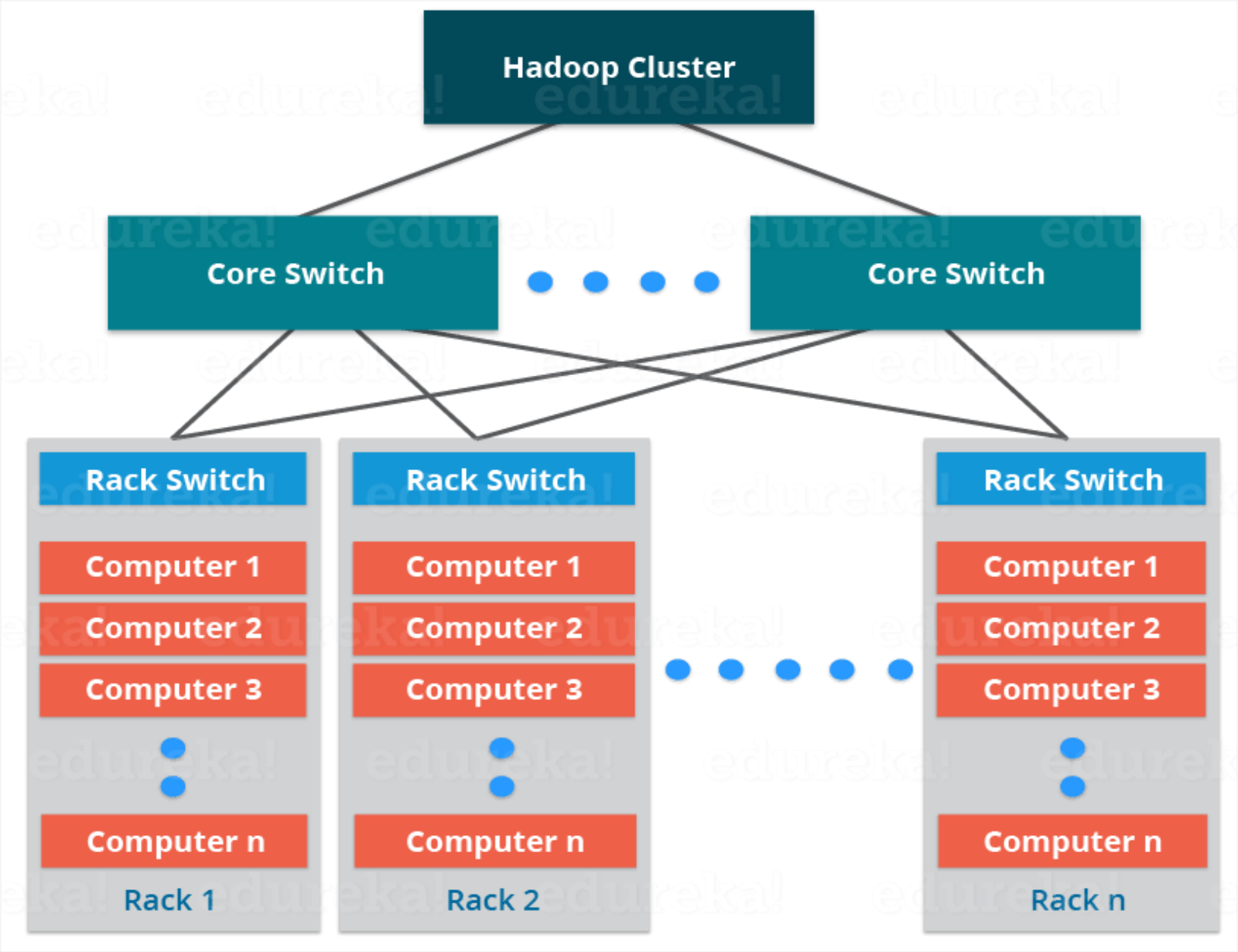
NameNode

- Initially, data is broken into abstract data blocks.
- The file metadata for these blocks, which include the file name, file permissions, IDs, locations, and the number of replicas, are stored in a fsimage, on the NameNode local memory.

If a NameNode fail, HDFS would not be able to locate any of the data sets distributed throughout the DataNodes. This makes the NameNode the single point of failure for the entire cluster. This vulnerability is resolved by implementing a Secondary NameNode or a Standby NameNode.

Secondary NameNode

- The Secondary NameNode served as the primary backup solution in early Hadoop versions.
- The Secondary NameNode, every so often, downloads the current ***fsimage*** instance and ***edit logs*** from the NameNode and merges them.
- The edited fsimage can then be retrieved and restored in the primary NameNode.
- The failover is not an automated process as an administrator would need to recover the data from the Secondary NameNode manually.


Hadoop Cluster
Core Switch
Core Switch
Rack Switch
Computer 1
Computer 2
Computer 3
Computer n
Rack 1
Rack Switch
Computer 1
Computer 2
Computer 3
Computer n
Rack 2
Rack Switch
Computer 1
Computer 2
Computer 3
Computer n
Rack n

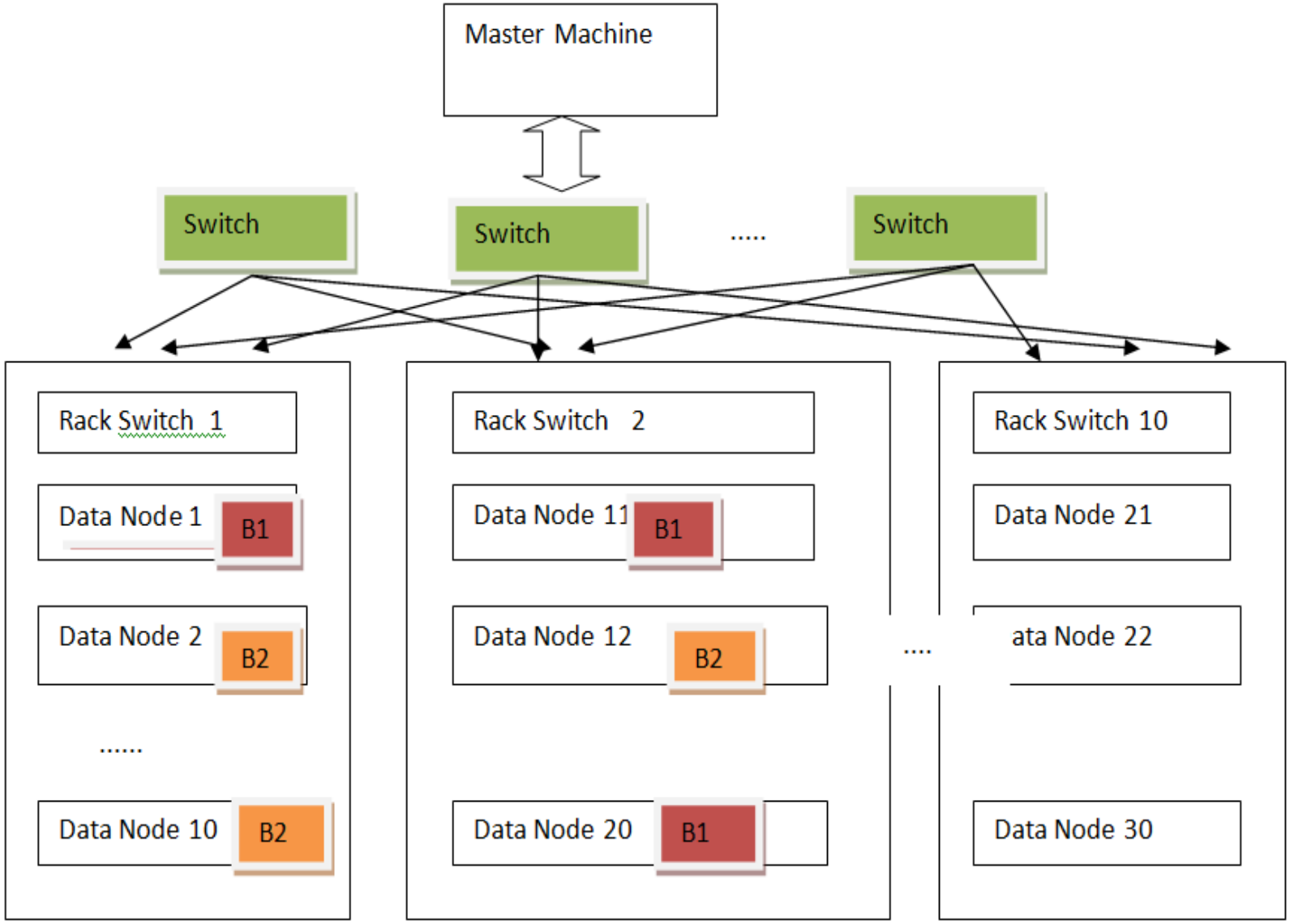
Rack Awareness Algorithm -> Multiple locations -> High availability

Replication factor-> 3

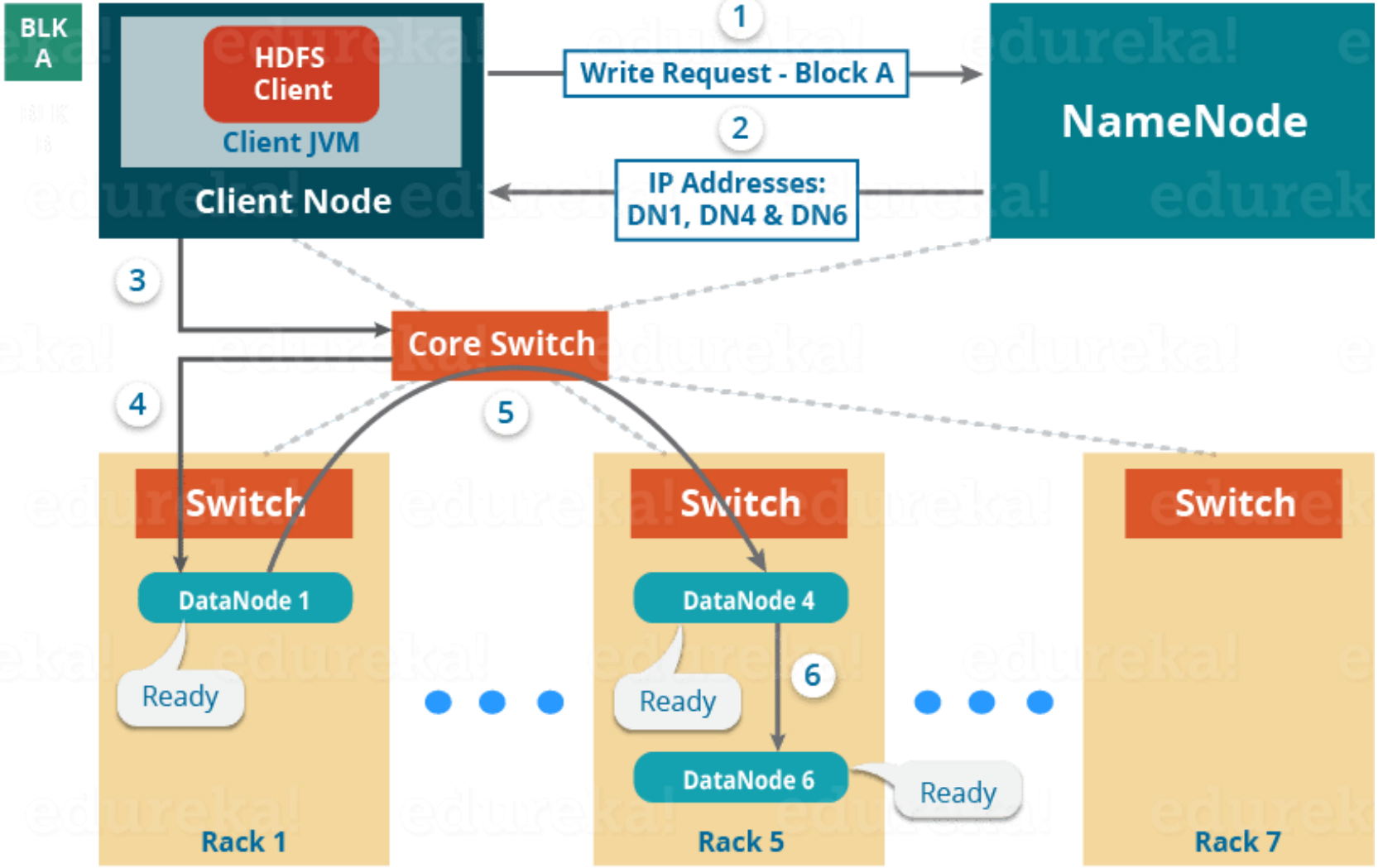
File -> Sample.txt (Size 520MB)

Blocks -> fixed sized blocks (Size - 64 MB) (size -> 128MB)

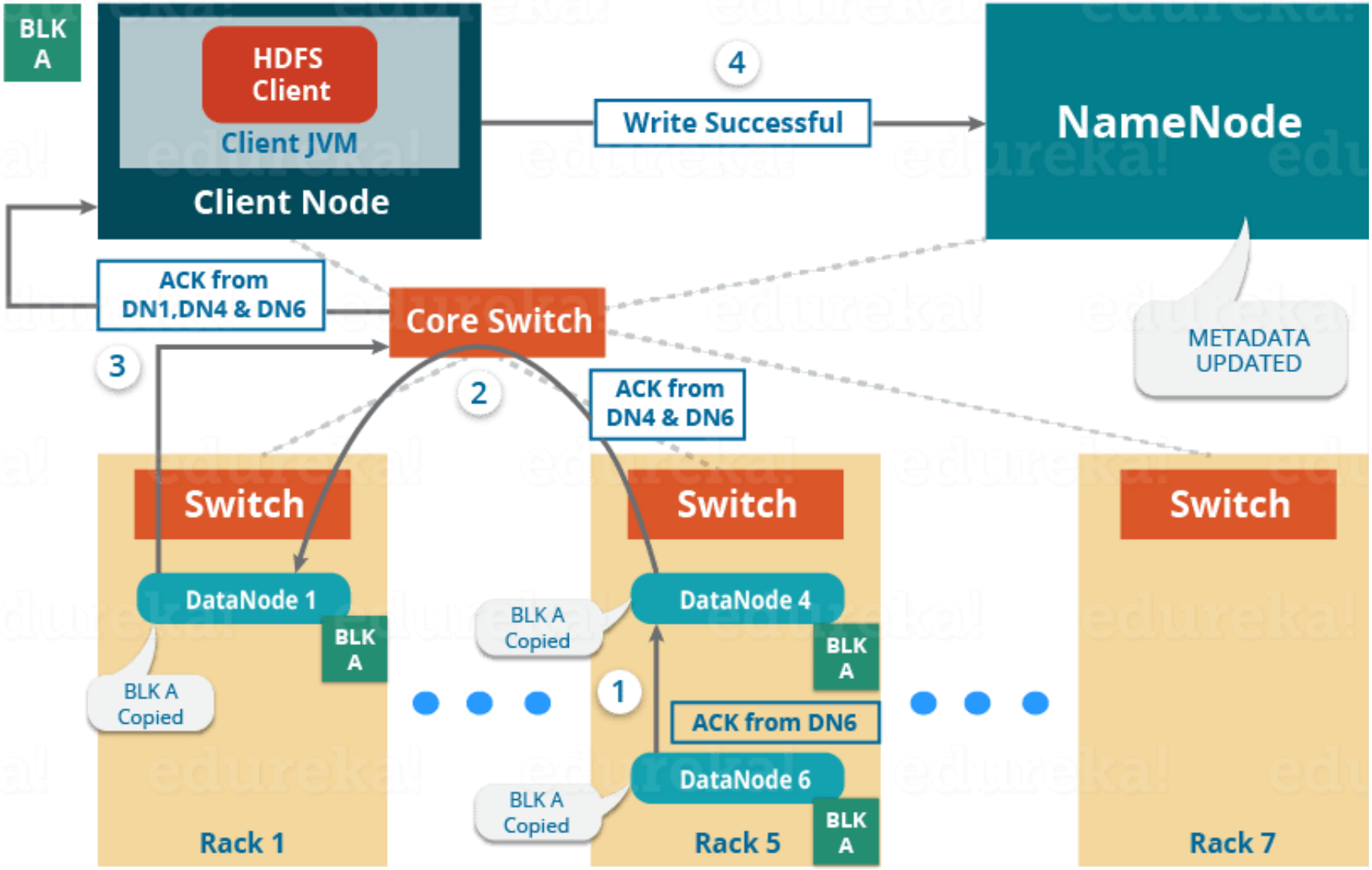
File -> Sample.txt (Size 520MB) 520/128 = 5


Master Machine
Switch
Switch
.....
Switch
Rack Switch 1
Data Node 1
B1
Data Node 2
B2
.....
Data Node 10
B2
Rack Switch 2
Data Node 11
B1
Data Node 12
B2
Data Node 20
B1
....
Rack Switch 10
Data Node 21
ata Node 22
Data Node 30

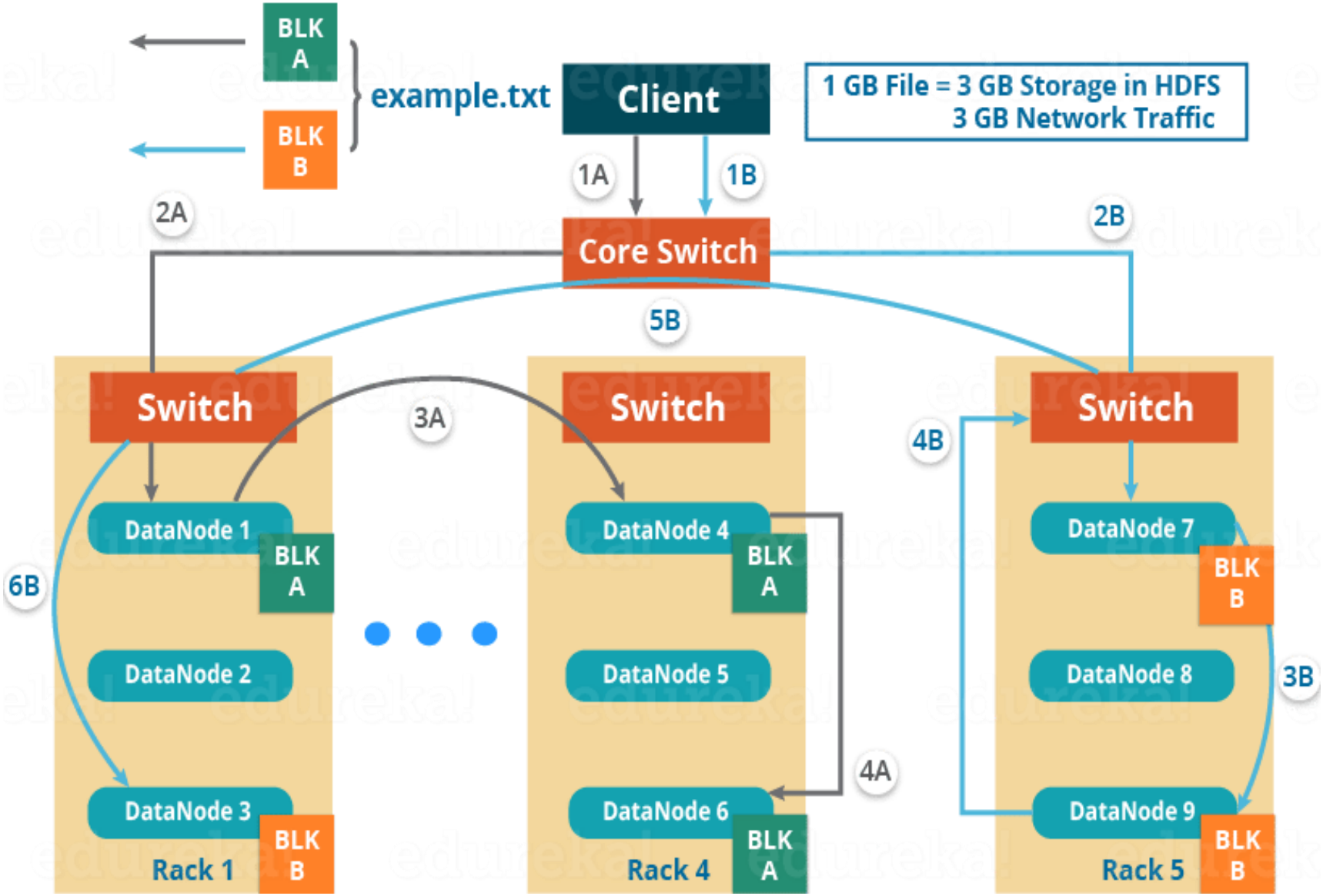
Setting up HDFS - Write Pipeline



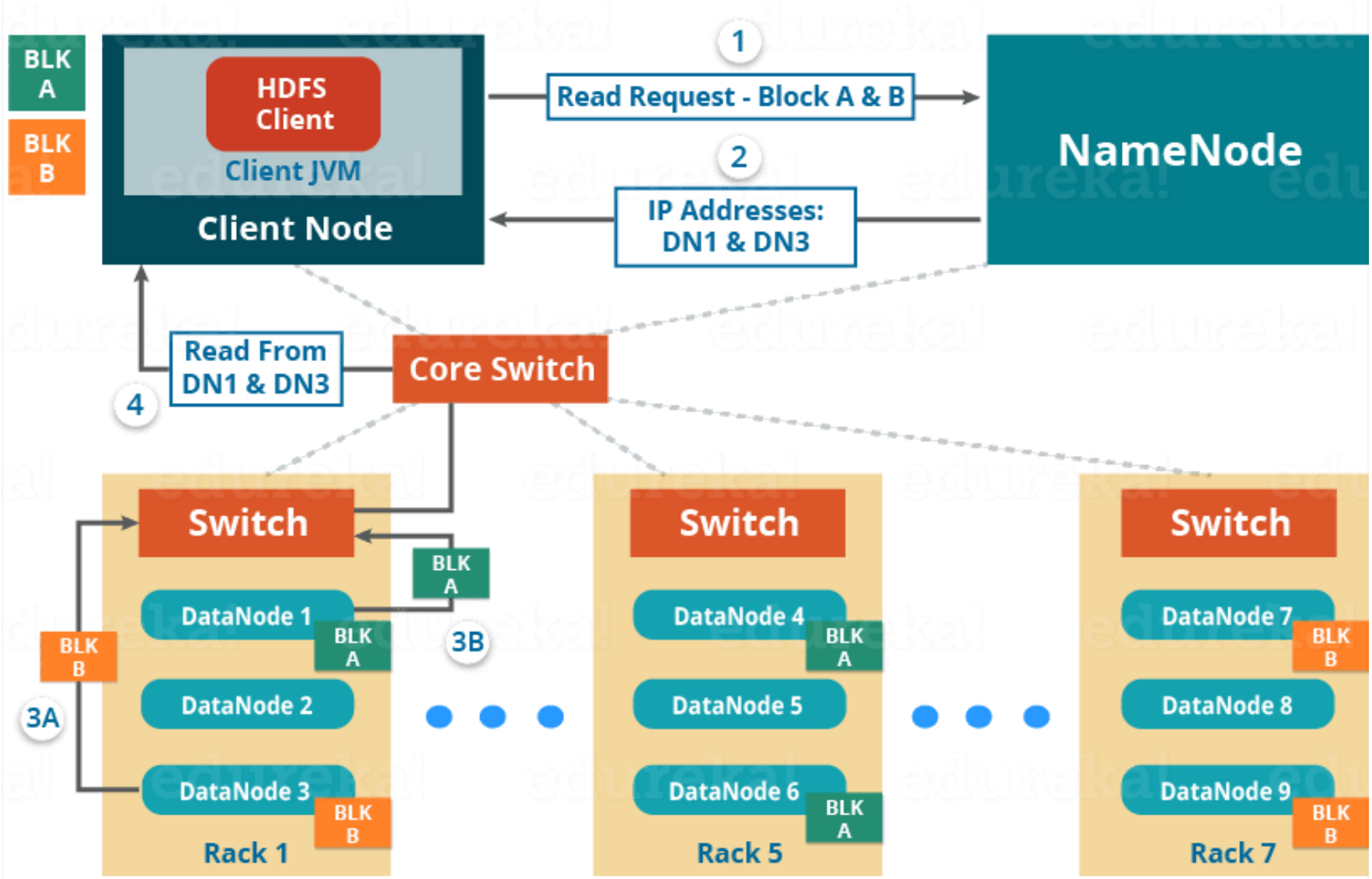
Acknowledgement in HDFS - Write



HDFS Multi - Block Write Pipeline



HDFS - Read Architecture



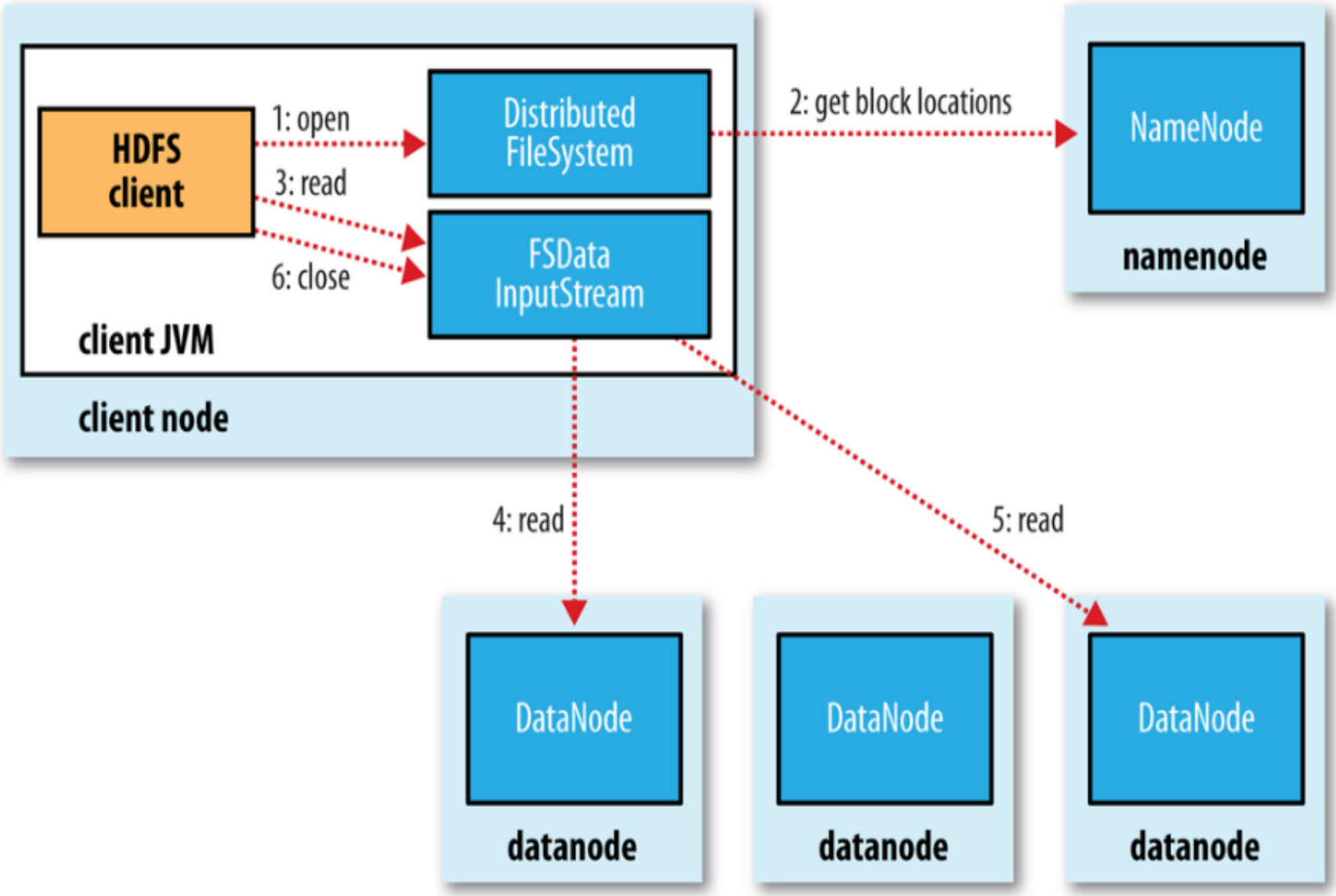


Figure 3-2. A client reading data from HDFS

Features of HDFS

Cost:

The HDFS, in general, is deployed on a commodity hardware like your desktop/laptop which you use every day.

So, it is very economical in terms of the cost of ownership of the project. Since, we are using low cost commodity hardware, you don't need to spend huge amount of money for scaling out your Hadoop cluster.

In other words, adding more nodes to your HDFS is cost effective.

Features of HDFS

Variety and Volume of Data:

When we talk about HDFS then we talk about storing huge data i.e. Terabytes & petabytes of data and different kinds of data.

So, you can store any type of data into HDFS, be it structured, unstructured or semi structured.

Features of HDFS

Reliability and Fault Tolerance:

When you store data on HDFS, it internally divides the given data into data blocks and stores it in a distributed fashion across your Hadoop cluster. The information regarding which data block is located on which of the data nodes is recorded in the metadata.

NameNode manages the meta data and the DataNodes are responsible for storing the data.Name node also replicates the data i.e. maintains multiple copies of the data.

This replication of the data makes HDFS very reliable and fault tolerant. So, even if any of the nodes fails, we can retrieve the data from the replicas residing on other data nodes. By default, the replication factor is 3. Therefore, if you store 1 GB of file in HDFS, it will finally occupy 3 GB of space. The name node periodically updates the metadata and maintains the replication factor consistent.

Features of HDFS

Data Integrity:

Data Integrity talks about whether the data stored in my HDFS are correct or not. HDFS constantly checks the integrity of data stored against its checksum.

If it finds any fault, it reports to the name node about it. Then, the name node creates additional new replicas and therefore deletes the corrupted copies.

Features of HDFS

High Throughput:

Throughput is the amount of work done in a unit time. It talks about how fast you can access the data from the file system. Basically, it gives you an insight about the system performance.

Ex: Over Single machine, Ten machines can collectively work to enhance computation. There we were able to reduce the processing time from 43 minutes to a mere 4.3 minutes as all the machines were working in parallel.

Therefore, by processing data in parallel, we decreased the processing time tremendously and thus, achieved high throughput.

Features of HDFS

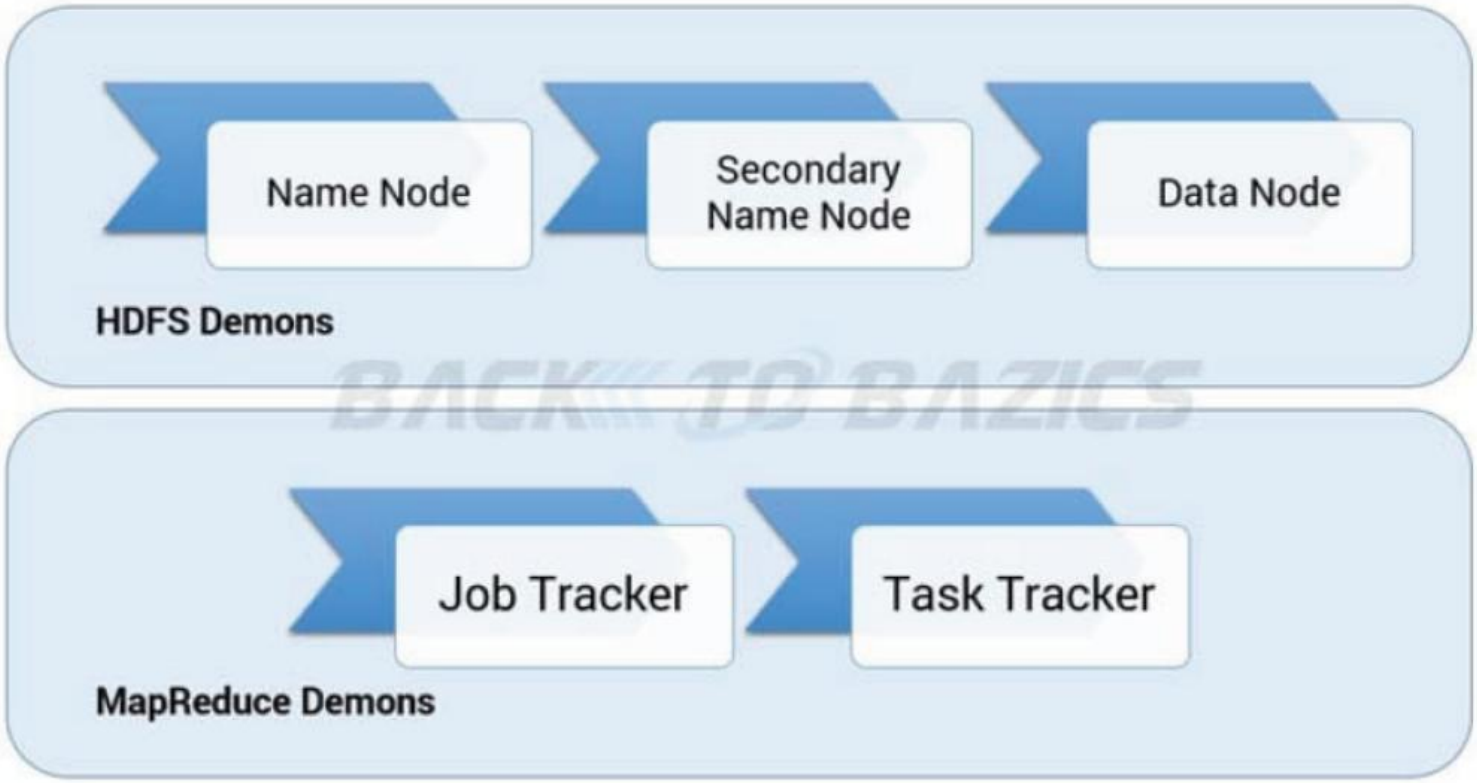
Data Locality:

Data locality talks about moving processing unit to data rather than the data to processing unit.

In our traditional system, we used to bring the data to the application layer and then process it.

But now, because of the architecture and huge volume of the data, bringing the data to the application layer will reduce the network performance to a noticeable extent.

So, in HDFS, we bring the computation part to the data nodes where the data is residing. Hence, you are not moving the data, you are bringing the program or processing part to the data.

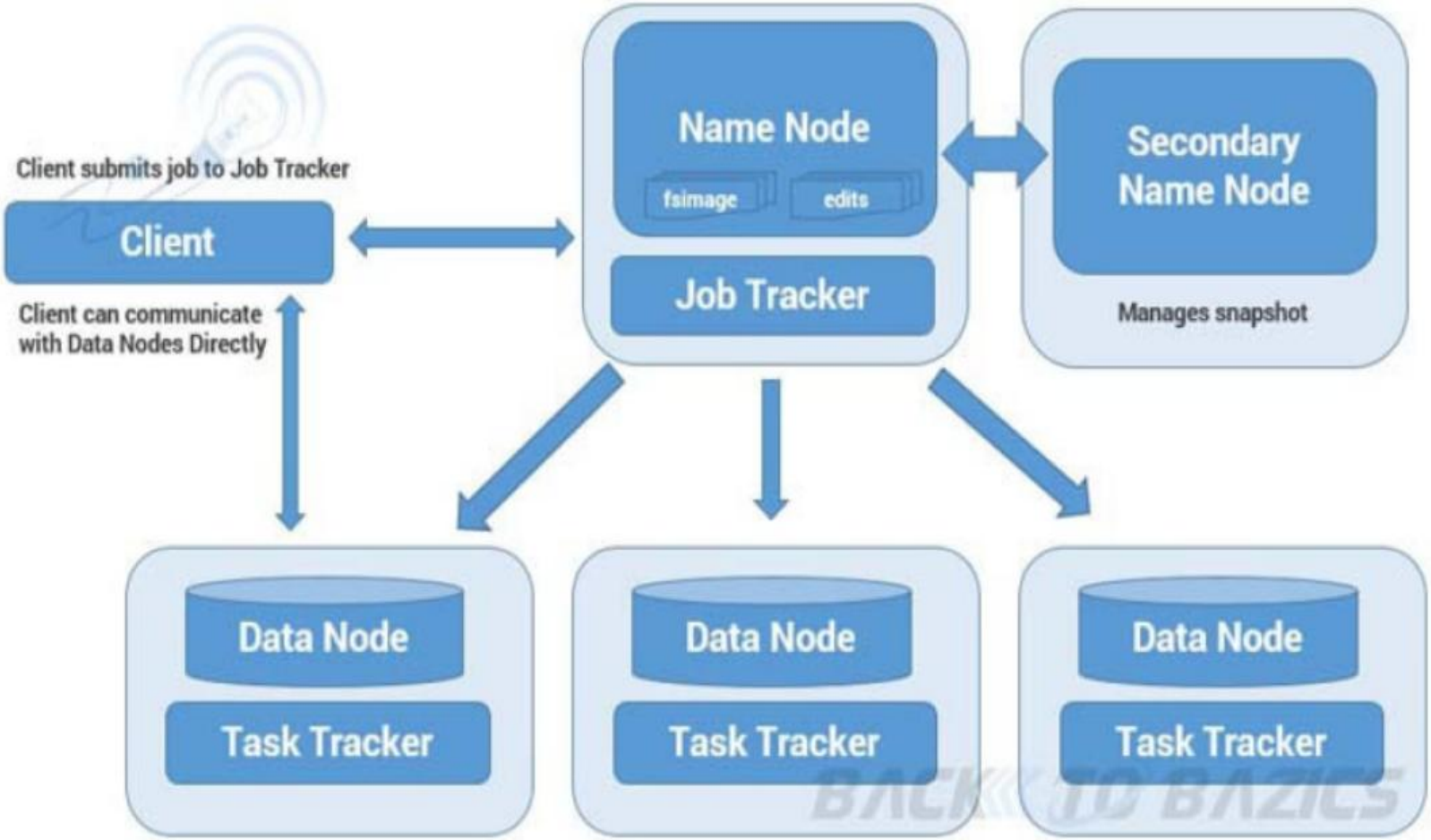


Hadoop 1.x Architecture Daemons

HDFS – Hadoop Distributed File System

HDFS in Hadoop 1.x mainly has 3 daemons which are

1. Name Node
2. Secondary Name Node
3. Data Node



Hadoop 1.x Architecture in Detail

Name Node

- There is only single instance of this process runs on a cluster and that is on a master node
- It is responsible for manage metadata about files distributed across the cluster
- It manages information like location of file blocks across cluster and it's permission
- This process reads all the metadata from a file named ***fsimage*** and keeps it in memory
- After this process is started, it updates metadata for newly added or removed files in RAM
- It periodically writes the changes in one file called ***edits*** as edit logs
- This process is a heart of HDFS, if it is down HDFS is not accessible any more

Secondary Name Node

- For this also, only single instance of this process runs on a cluster
- This process can run on a master node (for smaller clusters) or can run on a separate node (in larger clusters)depends on the size of the cluster
- One misinterpretation from name is “This is a backup Name Node” but IT IS NOT!!!!!
- It manages the metadata for the Name Node. In the sense, it reads the information written in edit logs (by NameNode) and creates an updated file of current cluster metadata
- Then it transfers that file back to Name Node so that fsimage file can be updated
- So, whenever Name Node daemon is restarted it can always find updated information in fsimage file

Data Node

- There are many instances of this process running on various slave nodes(referred as Data nodes)
- It is responsible for storing the individual file blocks on the slave nodes in Hadoop cluster
- Based on the replication factor, a single block is replicated in multiple slave nodes(only if replication factor is > 1) to prevent the data loss
- Whenever required, this process handles the access to a data block by communicating with Name Node
- This process periodically sends heart beat to Name Node to make Name Node aware that slave process is running

MapReduce – Distributed Computation

MapReduce component is responsible for distributed computing on Hadoop Cluster. Basically it uses two daemons named

1. Job Tracker
2. Task Tracker

Job Tracker

- Only one instance of this process runs on a master node same as Name Node
- Any MapReduce job is submitted to Job Tracker first
- Job Tracker checks for the location various file blocks used in MapReduce processing
- Than it initiates the separate tasks on various Data Nodes(where blocks are present) by communicating with TaskTracker Daemons

Task Tracker

- This process has multiple instances running on the slave nodes(typically runs on the slave nodes where Data Node process is running)
- It receives the job information from Job Tracker daemon and initiates a task on that slave node
- In most of the cases, Task Tracker initiates the task on the same node where there physical data block is present
- Same as Data Node daemon, this process also periodically sends heart beats to Job Tracker to make Job Tracker aware that slave process is running

Limitations of Hadoop 1.x Architecture

- Major drawback of Hadoop 1.x Architecture is Single Point of Failure as there is no backup Name Node.
- Job scheduling, resource management and job monitoring are being done by Job Tracker which is tightly coupled with Hadoop. So Job Tracker is not able to manage resources outside Hadoop.
- Job Tracker has to coordinate with all task tracker so in a very big cluster it will be difficult to manage huge number of task trackers altogether.
- Due to single Name Node, there is no concept of namespaces in Hadoop 1.x. So everything is being managed under single namespace.
- Using Hadoop 1.x architecture, Hadoop Cluster can be scaled upto ~4000 nodes. Scalablity beyond that may cause performance degradation and increasing task failure ratio.

Hadoop cluster architecture introduced in Hadoop 2,allows for two or more NameNodes running in the cluster in a hot standby configuration

Standby NameNode

The **High Availability** feature was introduced in Hadoop 2.0 and subsequent versions to avoid any downtime in case of the NameNode failure. This feature allows you to maintain two NameNodes running on separate dedicated master nodes. The Standby NameNode is an automated failover in case an Active NameNode becomes unavailable. The Standby NameNode additionally carries out the check-pointing process.

Zookeeper

Zookeeper is a lightweight tool that supports high availability and redundancy. A Standby NameNode maintains an active session with the Zookeeper daemon.

If an Active NameNode falters, the Zookeeper daemon detects the failure and carries out the failover process to a new NameNode. Use Zookeeper to automate failovers and minimize the impact a NameNode failure can have on the cluster.

CONFIGURING HADOOP CLUSTER

Operational modes of Hadoop

- Local (Standalone) mode
- Pseudo Distributed mode
- Fully Distributed mode

LOCAL OR STANDALONE MODE

- The standalone mode is the default mode for Hadoop.
- No daemons running, everything is runs in a single JVM.
- Has no DFS. All three XML files (or hadoopsite.xml before version 0.20) are empty under this default mode.
- There‘s no need to communicate with other nodes, the standalone mode doesn‘t use HDFS, nor will it launch any of the Hadoop daemons. Its primary use is for developing and debugging the application logic of a MapReduce program without the additional complexity of interacting with the daemons.

Properties:

(i) Default mode of Hadoop.
(ii) HDFS is not utilized in this mode.
(iii) Local File System is used for input and output.
(iv) Used for debugging purpose.
(v) Standalone is much faster than Pseudo-Distributed mode.

PSEUDO DISTRIBUTED MODE

- The pseudo-distributed mode is running Hadoop in a cluster of one with all daemons running on a single machine.
- The pseudo-distributed mode is also known as single node cluster where both Name node and Data node will reside on the same machine.
- Replication factor will be 1 for blocks.
- This mode complements the standalone mode for debugging your code, allowing you to examine memory usage, HDFS input/output issues, and other daemon interactions.

- **Properties:**

  (i) Configuration is required in given three files for this node.
  (ii) Here one node is used as Master/Data/Job Tracker/Task Tracker node.
  (iii) This is used for real code to test in HDFS.
  (iv) This is a cluster, where all daemons are running on one node itself.

FULLY DISTRIBUTED MODE

This is the production mode of Hadoop where multiple nodes will be running.
Data will be distributed across several nodes and processing will be done on each node.
Separate nodes for master and slave.
Hadoop daemons run on a cluster of machines.

- master—The master node of the cluster and host of the NameNode and Job- Tracker daemons
- backup—The server that hosts the Secondary NameNode daemon
- hadoop1, hadoop2, hadoop3, ...—The slave boxes of the cluster running both DataNode and TaskTracker daemons

Properties:

(i) This is a production phase.
(ii) Data are used and distributed across many nodes.
(iii) Different nodes will be used as Master node/Data node..etc

CONFIGURATION FILES

- hadoop-env.sh
- core-site.xml
- hdfs-site.xml
- mapred-site.xml

The Hadoop-env.sh, core-ite.xml, hdfs-site.xml, mapred-site.xml, Masters and Slaves are all available under ‘conf’ directory of Hadoop installation directory.

hdfs-site.xml

- The hdfs-site.xml file contains the configuration settings for HDFS daemons; the NameNode, the Secondary NameNode, and the DataNodes.
- Here, we can configure hdfs-site.xml to specify default block replication and permission checking on HDFS.
- The actual number of replications can also be specified when the file is created.
- The default is used if replication is not specified in create time.

hdfs-site.xml

Location: /etc/Hadoop/hdfs-site.xml

→ **To configure the replication factor**

```
<property>
<name> dfs.replication </name>
<value> 3</value>
</property>
```

→ **To configure the namenode metadata location**

```
<property>
<name> dfs.namenode.name.dir </name>
<value> file:/user/local/hadoopdata/hdfs/namenode</value>
</property>
```

→ **To configure the datanode data storage location**

```
<property>
<name> dfs.datanode.name.dir </name>
<value> file:/user/local/hadoopdata/hdfs/datanode</value>
</property>
```

core-site.xml

->informs Hadoop daemon where NameNode runs in the cluster.

Configuration settings for Hadoop core such as I/O settings that are common to Hadoop and MapReduce.

-> hdfs master i.e., name node(IP address and port number)

Location: /etc/hadoop/core-site.xml

```
<property>
<name> fs.default.name</name>
<value> hdfs://IPAddress:9090</value>
</property>
```

mapred-site.xml

The mapred-site.xml file contains the configuration settings for MapReduce daemons; the job tracker and the task-trackers.

Configures the host name (IP address) and port of Mapreduce job tracker.

Location: /etc/Hadoop/mapred-site.xml

```
<property>
<name> mapred.job.tracker</name>
<value> IPAddress:9090</value>
</property>
```

mapred-site.xml

Property	Value	Description
mapred.job.tracker	<value> localhost:8021 </value>	The hostname and the port that the jobtracker RPC server runs on. If set to the default value of local, then the jobtracker runs in-process on demand when you run a MapReduce job.
mapred.local.dir	${hadoop.tmp.dir}/mapred/local	A list of directories where MapReduce stores intermediate data for jobs. The data is cleared out when the job ends.
mapred.system.dir	${hadoop.tmp.dir}/mapred/system	The directory relative to fs.default.name where shared files are stored, during a job run.
mapred.tasktracker.map.tasks.maximum	2	The number of map tasks that may be run on a tasktracker at any one time
mapred.tasktracker.reduce.tasks.maximum	2	The number of reduce tasks tat may be run on a tasktracker at any one time.

hadoop-env.sh, masters and slaves files

-> Evironmental variables that are used in the scripts to run Hadoop

Location : /etc/hadoop/hadoop-env.sh

Export JAVA HOME”c:/programfiles/Java/….

-> Masters file: contains IP address of node where secondary name node is running.

-> Slaves file: contains the list of nodes where data nodes and tasks are running.

Format namenode

-> To format the file system, run the command

hadoop namenode –format

-> Starting and stopping single node cluster

Start –all.sh

(This will start up a namenode, secondary namenode, data node, job tracker and task tracker in the cluster

jps -> list of all processes ID (java process status)